

Abdullah Alrayyis - 23060021
Rasha Muhammed Ali - 22061010
Habib Sultani - 22061001
Saad Mohammed Abderezak - 23061761

Ders: BİL 304 İşletim Sistemleri

Öğretim Üyesi: Doç. Dr. Sercan DEMİRCİ

Dönem: 2025/2026 Bahar

GitHub linki → <https://github.com/Habibsultani/rpl-udp>

Contiki-NG Üzerinde OTA Firmware Aktarımı ve ELF Analizi

1. Giriş

Bu çalışmada Contiki-NG işletim sistemi üzerinde çalışan düğümler arasında OTA yani Over-The-Air firmware aktarımı gerçekleştirilmiştir. Projenin amacı, gönderici düğümde bulunan güncel firmware dosyasının küçük bloklara ayrılarak kablosuz ağ üzerinden alıcı düğüme güvenilir şekilde gönderilmesidir.

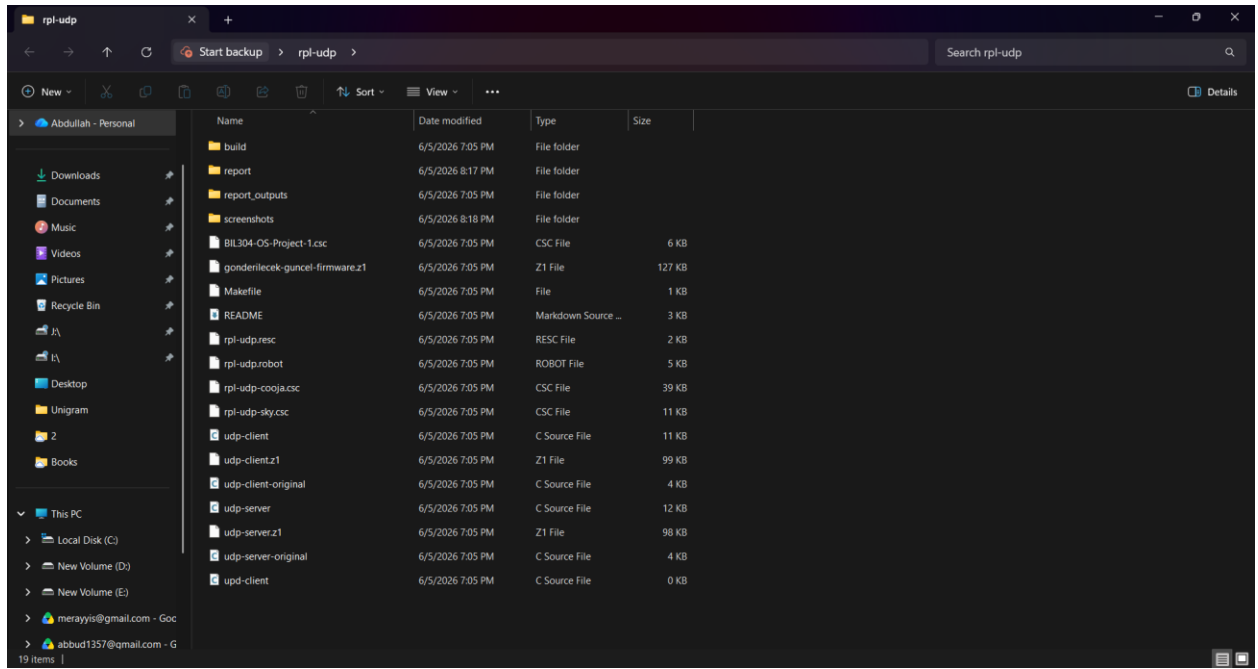
Çalışma Cooja simülatörü üzerinde gerçekleştirilmiştir. Simülasyonda üç düğüm bulunmaktadır. ID 2 numaralı düğüm gönderici, ID 1 numaralı düğüm alıcı, ID 3 numaralı düğüm ise ara komşu/iletici görevindedir. Gönderici düğüm firmware dosyasını bloklara ayırmış, her bloğu sıra numarası ve checksum bilgisi ile göndermiştir. Alıcı düğüm ise aldığı blokları kontrol ederek kalıcı flash alanına yazmış ve aktarım sonunda tüm firmware imajının başarıyla alındığını doğrulamıştır.

Bu raporda sadece 1. aşama: OTA geliştirme süreci ve 2. aşama: ELF analizi ele alınmıştır. 3. aşama olan CC1352R gerçek donanım üzerinde reboot/bootloader uyarlaması bu çalışma kapsamında yapılmamıştır.

2. Proje Dosya Yapısı

Projede kullanılan temel dosyalar şunlardır:

- `udp-client.c` → OTA gönderici düğüm tarafında kullanılan kaynak kod.
- `udp-server.c` → OTA alıcı düğüm tarafında kullanılan kaynak kod.
- `gonderilecek-guncel-firmware.z1` → OTA ile gönderilecek firmware imajı.
- `BIL304-OS-Project-1.csc` → Cooja simülasyon senaryosu.
- `Makefile` → Derleme işlemleri için kullanılan dosya.
- `build/` → Derleme sonrası oluşan çıktıların bulunduğu klasör.
- `screenshots/` → Rapor için alınan ekran görüntülerinin bulunduğu klasör.



Şekil 1. Proje klasör yapısı ve kullanılan dosyalar

Şekil 1’de proje klasörünün son hali gösterilmiştir. Projede OTA aktarımı için gerekli olan kaynak kod dosyaları, Cooja simülasyon dosyası, gönderilecek firmware dosyası ve derleme çıktıları bulunmaktadır.

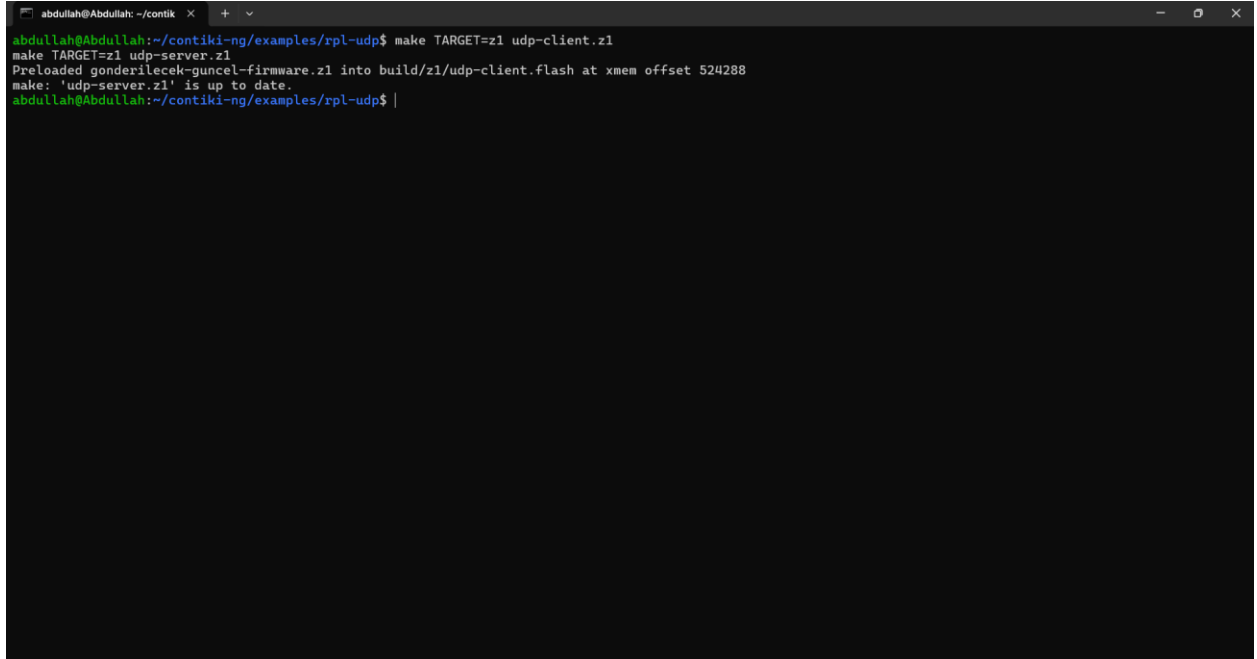
3. Derleme Süreci

Projede gönderici ve alıcı düğümlerin çalıştırılabilmesi için udp-client.z1 ve udp-server.z1 dosyaları derlenmiştir. Derleme işlemi WSL terminali üzerinden Contiki-NG examples/rpl-udp dizininde yapılmıştır.

Kullanılan komut:

```
make TARGET=z1 udp-client.z1
make TARGET=z1 udp-server.z1
```

Derleme sırasında gonderilecek-guncel-firmware.z1 dosyası udp-client.flash içine belirli bir offset değerinden itibaren yerleştirilmiştir. Terminal çıktısında görüldüğü üzere firmware dosyası build/z1/udp-client.flash içerisine xmem offset 524288 adresinden itibaren eklenmiştir.



```
abduallah@Abduallah: ~/contiki x + v
abduallah@Abduallah:~/contiki-ng/examples/rpl-udp$ make TARGET=z1 udp-client.z1
make TARGET=z1 udp-server.z1
Preloaded gonderilecek-guncel-firmware.z1 into build/z1/udp-client.flash at xmem offset 524288
make: 'udp-server.z1' is up to date.
abduallah@Abduallah:~/contiki-ng/examples/rpl-udp$ |
```

Şekil 2. Projenin başarılı şekilde derlenmesi

Şekil 2’de udp-client.z1 ve udp-server.z1 dosyalarının başarılı şekilde derlendiği görülmektedir. Ayrıca gönderilecek güncel firmware dosyasının udp-client.flash içerisine eklendiği görülmektedir. Bu işlem, gönderici düğümün firmware imajına erişebilmesi için gereklidir.

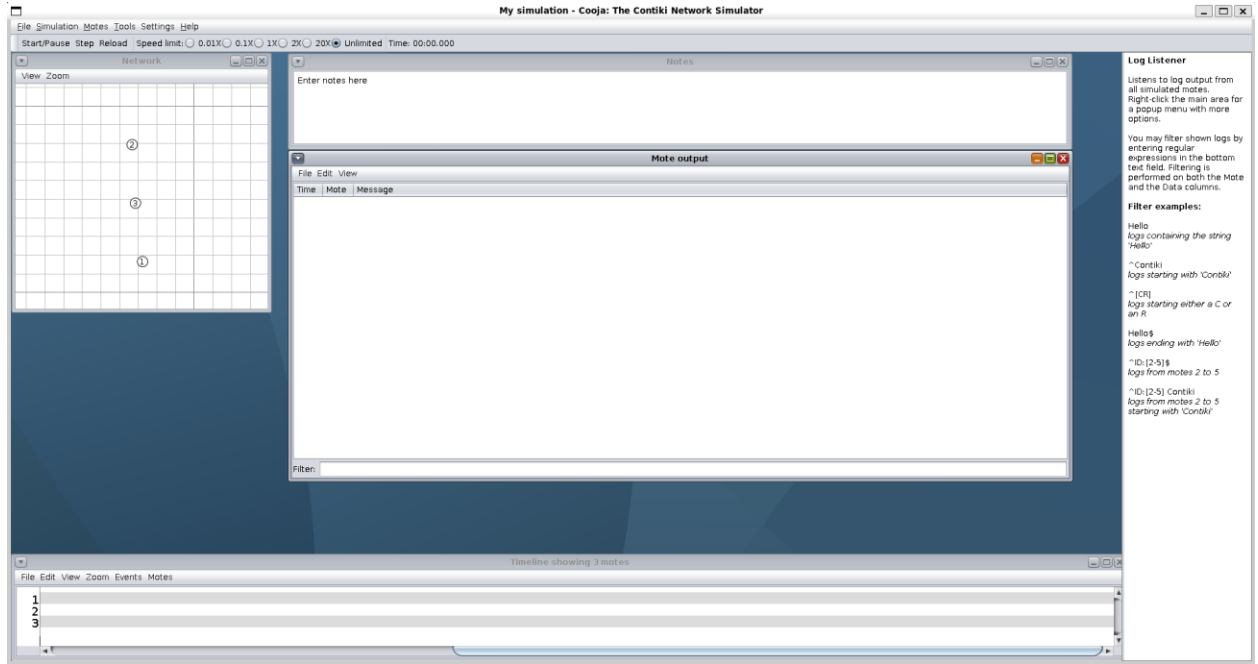
4. Cooja Simülasyon Ortamı

Simülasyon Cooja üzerinde BIL304-OS-Project-1.csc dosyası açılarak başlatılmıştır.

Senaryoda üç düğüm yer almaktadır:

ID 1	Alıcı düğüm / OTA server
ID 2	Gönderici düğüm / OTA client
ID 3	Komşu / iletici düğüm

Başlangıçta simülasyon durdurulmuş durumdadır. Bu aşamada düğümlerin Cooja ortamında doğru şekilde yüklendiği kontrol edilmiştir.



Şekil 3. Cooja simülasyon senaryosunun açılması

Şekil 3'te Cooja simülatörü üzerinde üç düğümlü OTA senaryosu görülmektedir. ID 2 gönderici düğüm, ID 1 alıcı düğüm ve ID 3 ara komşu düğüm olarak kullanılmıştır.

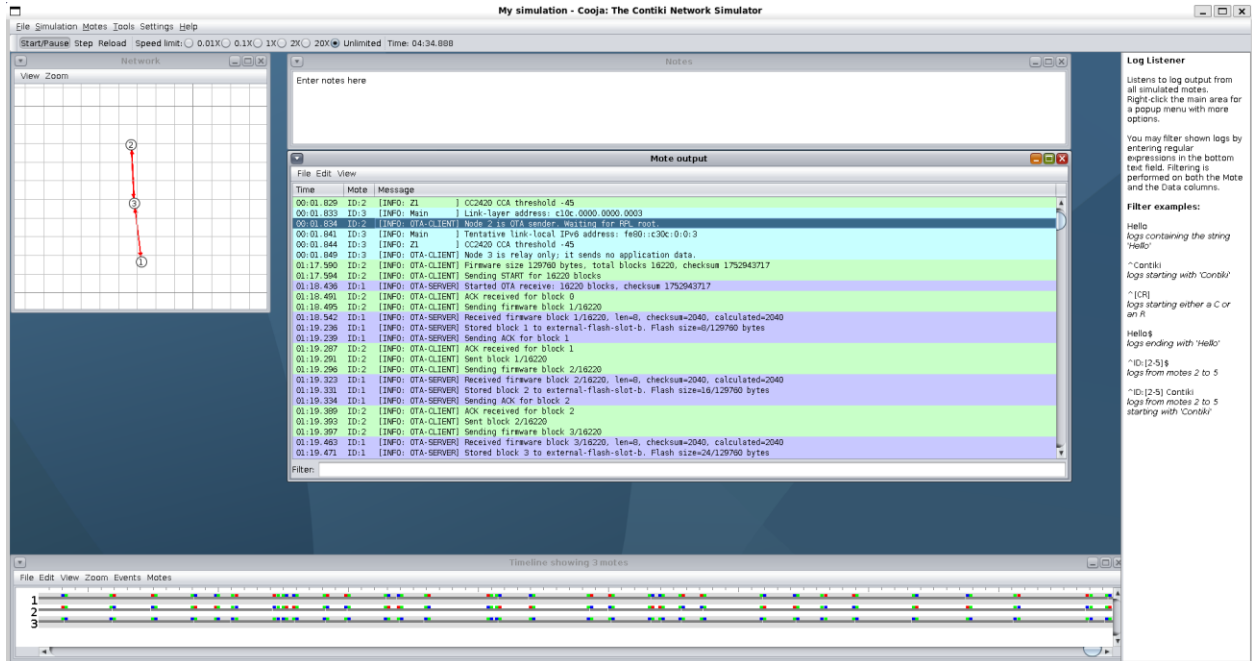
5. OTA Aktarım Mekanizması

Bu projede firmware dosyası tek parça halinde gönderilmemiştir. Bunun yerine dosya küçük bloklara ayrılmıştır. Her blok için şu bilgiler taşınmıştır:

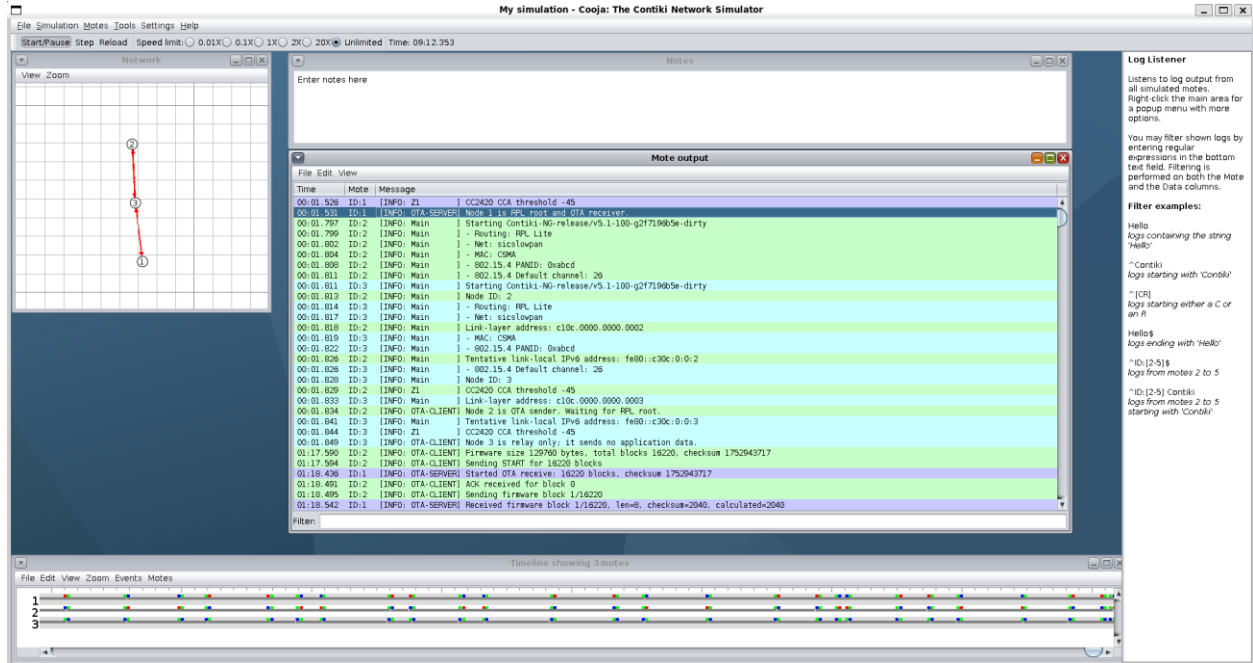
- Blok numarası
- Veri uzunluğu
- Checksum değeri
- Blok verisi

Gönderici düğüm önce START paketi göndererek OTA aktarımını başlatmıştır. Bu pakette toplam firmware boyutu, toplam blok sayısı ve tüm firmware için checksum bilgisi bulunmaktadır. Alıcı düğüm bu bilgileri aldıktan sonra gelen blokları kabul etmeye başlamıştır.

Log çıktısında firmware boyutunun **129760 byte**, toplam blok sayısının ise **16220** olduğu görülmektedir. Her blok 8 byte veri taşımaktadır.



Şekil 4. ID 2 numaralı düğümün OTA gönderici olması



Şekil 5. ID 1 numaralı düğümün OTA alıcı olması

Şekil 4'te ID 2 numaralı düğümün OTA gönderici olarak çalıştığı, şekil 5'te ise ID 1 numaralı düğümün OTA alıcı olarak çalıştığı görülmektedir. Gönderici düğüm firmware boyutunu 129760 byte ve toplam blok sayısını 16220 olarak belirlemiştir. Ardından START paketi gönderilmiş ve alıcı düğüm OTA aktarımını başlatmıştır.

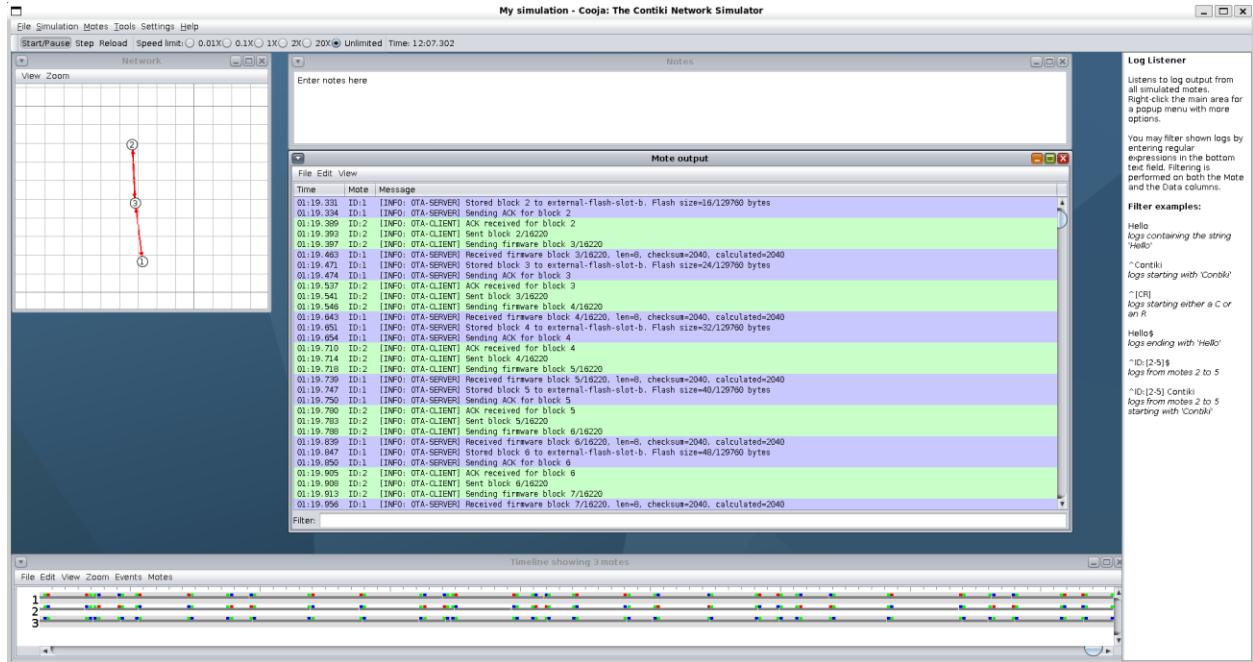
6. Blokların Gönderilmesi ve ACK Mekanizması

Firmware aktarımında güvenilirlik sağlamak için ACK tabanlı bir yöntem kullanılmıştır. Gönderici düğüm bir bloğu gönderdikten sonra alıcı düğümden ACK cevabı beklemektedir. Alıcı düğüm bloğu doğru şekilde aldığında ilgili blok için ACK mesajı göndermektedir.

Örneğin loglarda şu akış görülmektedir:

- Gönderici blok gönderir.
- Alıcı blok numarasını, uzunluğunu ve checksum değerini kontrol eder.
- Alıcı bloğu flash alanına yazar.
- Alıcı ACK gönderir.
- Gönderici ACK alınca bir sonraki bloğa geçer.

Bu yöntem stop-and-wait mantığına benzemektedir. Her blok onaylandıktan sonra sıradaki blok gönderildiği için aktarım kontrollü şekilde ilerlemektedir.



Şekil 6. İlk blokların gönderilmesi ve ACK alınması

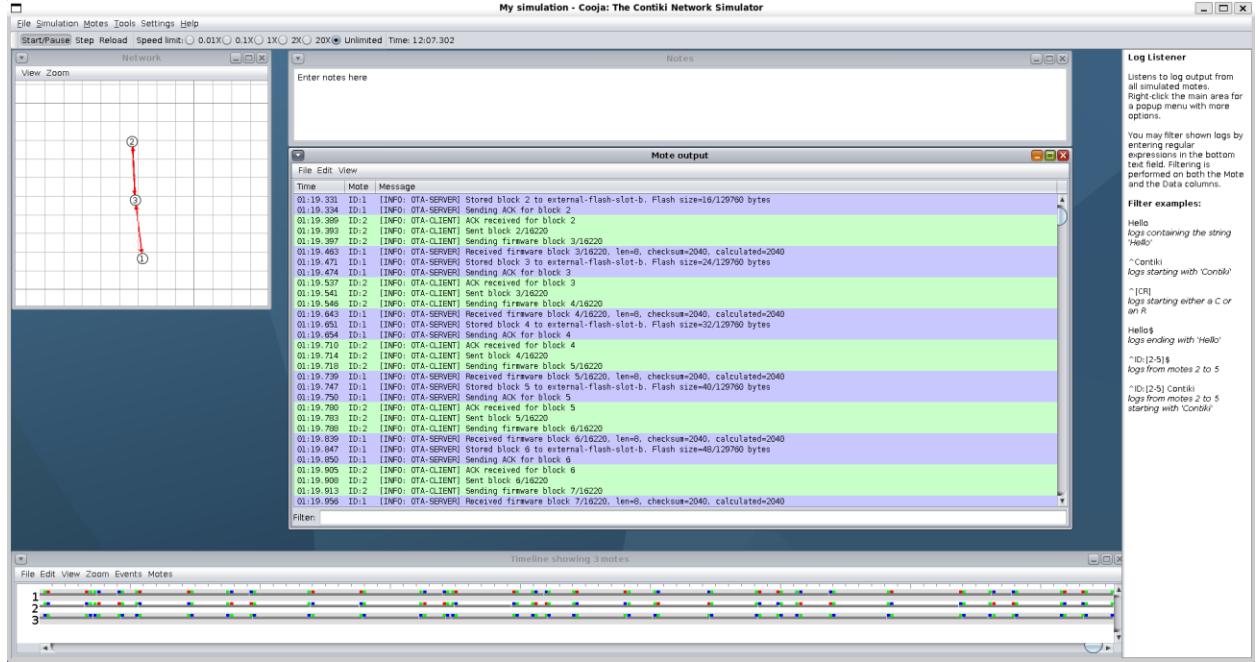
Şekil 6'te ilk blokların gönderildiği, alıcı düğüm tarafından alındığı, checksum kontrolünden geçirildiği ve flash alanına yazıldığı görülmektedir. Her doğru alınan bloktan sonra alıcı düğüm ACK cevabı göndermiştir.

7. Alıcı Düğümde Kalıcı Depolama

Alıcı düğüm gelen her firmware bloğunu sırayla flash benzeri kalıcı alana yazmaktadır. Log çıktılarında Stored block ... to external-flash-slot-b mesajları görülmektedir. Bu mesajlar, alınan blokların alıcı tarafında saklandığını göstermektedir.

Her blok yazıldıktan sonra flash üzerinde saklanan toplam veri boyutu artırılmaktadır.

Örneğin ilk bloklardan sonra flash boyutu 8, 16, 24-byte şeklinde artmaktadır. Bu durum, firmware imajının blok blok birleştirildiğini göstermektedir.

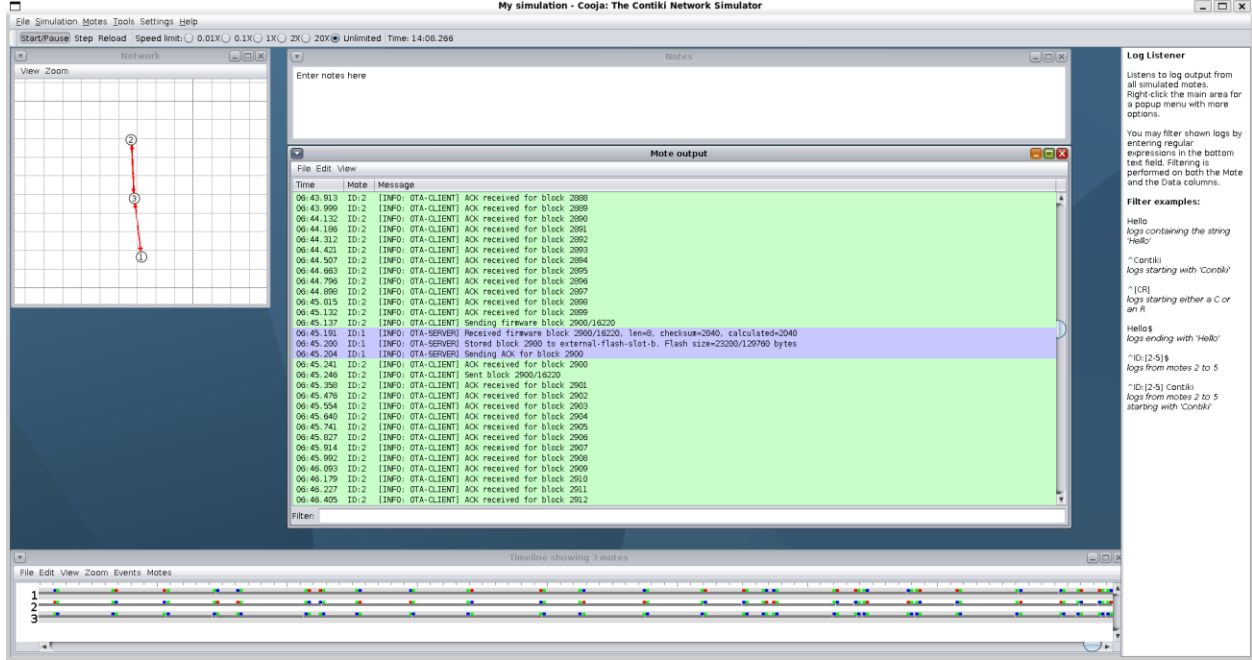


Şekil 7. Blokların alıcı tarafta flash alanına yazılması

Şekil 7’da gelen blokların alıcı düğüm tarafından external-flash-slot-b alanına yazıldığı görülmektedir. Flash boyutu her bloktan sonra artmaktadır. Bu durum, firmware imajının alıcı tarafta kalıcı olarak saklandığını göstermektedir.

8. Aktarımın Devam Etmesi ve Durum Yönetimi

Aktarım sırasında alıcı ve gönderici düğümler hangi bloğun gönderildiğini ve hangi bloğun onaylandığını takip etmektedir. Loglarda orta kısımlarda 2900. blok civarında aktarımın devam ettiği görülmektedir. Bu, sistemin sadece ilk birkaç bloğu değil, tüm firmware dosyasını sırayla gönderebildiğini göstermektedir.



Şekil 8. Aktarımın orta aşaması

Şekil 8'de aktarımın 2900. blok civarında devam ettiği görülmektedir. Aktarım sırasında çok fazla log oluşmaması için sistemde her blok için ayrıntılı log yazdırmak yerine, belirli aralıklarla yani yaklaşık her 100 gönderimde bir log yazdırılmıştır. Gönderici düğüm blokları sırayla göndermekte, alıcı düğüm ise bu blokları alıp flash alanına yazmakta ve ACK cevabı göndermektedir. Bu durum, aktarım sürecinin kontrollü ilerlediğini ve durum yönetiminin başarılı şekilde çalıştığını göstermektedir.

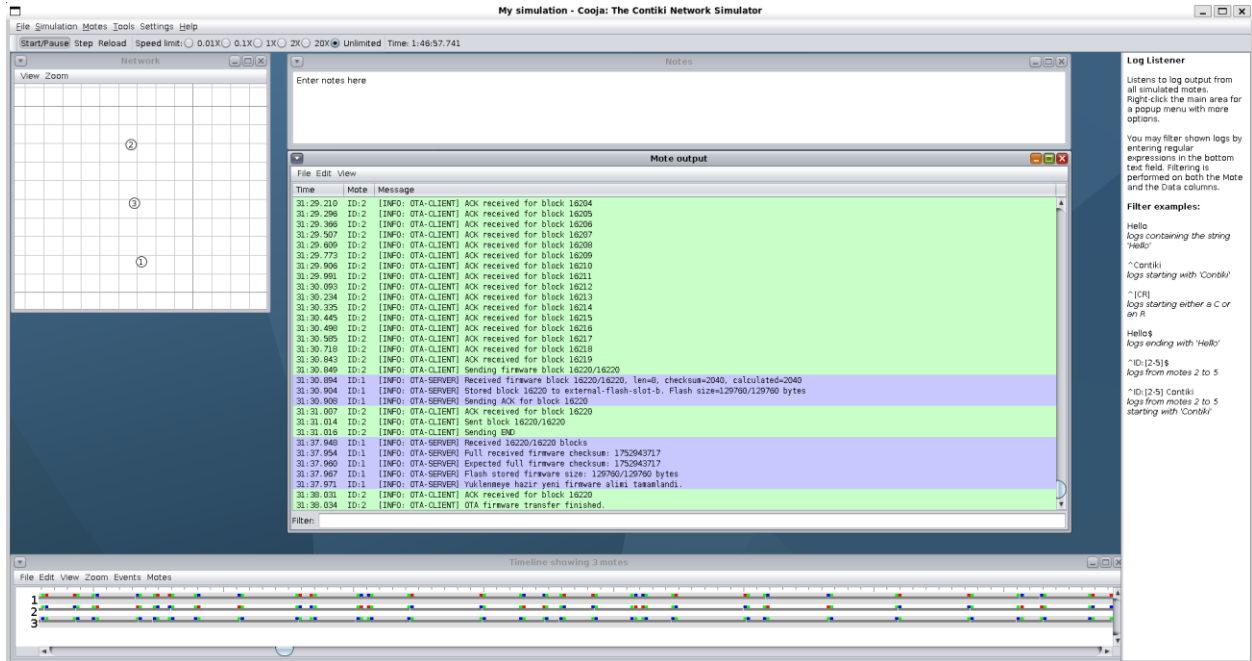
9. Aktarımın Tamamlanması ve Bütünlük Doğrulaması

Aktarım sonunda son blok olan 16220. blok da alıcı düğüm tarafından alınmıştır. Alıcı düğüm tüm blokların alındığını belirlemiş ve toplam firmware checksum değerini hesaplamıştır.

Log çıktısında şu bilgiler görülmektedir:

- Received 16220/16220 blocks
- Full received firmware checksum: 1752943717
- Expected full firmware checksum: 1752943717
- Flash stored firmware size: 129760/129760 bytes
- Yuklenmeye hazır yeni firmware alımı tamamlandı.
- OTA firmware transfer finished.

Bu çıktılar, firmware dosyasının eksiksiz alındığını, beklenen checksum ile hesaplanan checksum değerinin aynı olduğunu ve dosyanın alıcı tarafta başarıyla saklandığını göstermektedir.



Şekil 9. OTA aktarımının tamamlanması ve checksum doğrulaması

Şekil 9'de OTA aktarımının başarıyla tamamlandığı görülmektedir. Alıcı düğüm 16220 bloğun tamamını almış, flash alanında 129760/129760 byte veri saklanmıştır. Ayrıca beklenen checksum değeri ile alıcı tarafından hesaplanan checksum değeri aynı çıkmıştır. Bu nedenle firmware imajının bozulmadan aktarıldığı doğrulanmıştır.

10. Yeniden Gönderim Stratejisi

Projede güvenilir aktarım için ACK tabanlı bir yeniden gönderim yaklaşımı kullanılmıştır. Gönderici düğüm bir bloğu gönderdikten sonra belirli süre içinde ACK alamazsa aynı bloğu tekrar gönderecek şekilde tasarlanmıştır. Bu yapı, kablosuz ortamda oluşabilecek paket kaybı veya zaman aşımı durumlarına karşı güvenilirlik sağlamaktadır.

Bu çalışmada aktarımın sonunda tüm bloklar başarıyla alındığı için loglarda kalıcı bir başarısızlık görülmemiştir. Ancak sistemin temel mantığı, her bloğun ACK ile onaylanması ve ACK gelmediği durumda ilgili bloğun tekrar gönderilmesi üzerine kurulmuştur.

11. ELF Analizi

Gönderilen `gonderilecek-guncel-firmware.z1` dosyası ham binary dosyası değildir. Bu dosya MSP430 mimarisi için üretilmiş ELF biçiminde çalıştırılabilir firmware imajıdır. Bu nedenle dosyanın yapısını incelemek için MSP430 araç zinciri kullanılmıştır.

Kullanılan temel komutlar:

```
file gönderilecek-guncel-firmware.z1
msp430-readelf -h gönderilecek-guncel-firmware.z1
msp430-readelf -S gönderilecek-guncel-firmware.z1
msp430-size gönderilecek-guncel-firmware.z1
msp430-nm -n gönderilecek-guncel-firmware.z1
```

11.1. ELF Analizi

`msp430-readelf -h` komutu ile dosyanın ELF başlığı incelenmiştir. Bu çıktıda dosyanın ELF sınıfı, hedef mimarisi, giriş adresi ve dosya tipi görülmektedir.

```
abdullah@Abdullah: ~/contiki x + v
abdullah@Abdullah:~/contiki-ng/examples/rpl-udp$ msp430-readelf -h gönderilecek-guncel-firmware.z1
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 ff 00 00 00 00 00 00 00 00
  Class:             ELF32
  Data:              2's complement, little endian
  Version:           1 (current)
  OS/ABI:            Standalone App
  ABI Version:       0
  Type:              EXEC (Executable file)
  Machine:           Texas Instruments msp430 microcontroller
  Version:           0x1
  Entry point address: 0x3100
  Start of program headers: 52 (bytes into file)
  Start of section headers: 94584 (bytes into file)
  Flags:             0x10000001
  Size of this header: 52 (bytes)
  Size of program headers: 32 (bytes)
  Number of program headers: 6
  Size of section headers: 40 (bytes)
  Number of section headers: 21
  Section header string table index: 18
abdullah@Abdullah:~/contiki-ng/examples/rpl-udp$ |
```

ELF başlık çıktısında firmware dosyasının MSP430 mimarisi için hazırlanmış çalıştırılabilir bir ELF dosyası olduğu görülmektedir. Bu bilgi, gönderilen dosyanın yalnızca ham veri değil, belirli bir mimari için derlenmiş firmware imajı olduğunu göstermektedir.

11.2. Section Analizi

```
msp430-readelf -S
```

komutu ile firmware dosyasındaki bölümler incelenmiştir. ELF dosyasında genel olarak şu bölümler bulunur:

- `.text` → Program komutlarının bulunduğu bölümdür.
- `.data` → Başlangıç değeri olan global/statik değişkenlerin bulunduğu bölümdür.
- `.bss` → Başlangıç değeri olmayan değişkenlerin bulunduğu bölümdür.
- `.rodata` → Sabit/veri okuma amaçlı bilgilerin bulunduğu bölümdür.
- `.vectors` → Kesme vektörleri ve başlangıç adresiyle ilişkili bilgileri içerir.

```
abdullah@Abdullah: ~/contik
abdullah@Abdullah:~/contiki-ng/examples/rpl-udp$ msp430-readelf -S gonderilecek-guncel-firmware.z1
There are 21 section headers, starting at offset 0x17178:

Section Headers:
[Nr] Name                Type              Addr      Off      Size    ES Flg Lk Inf Al
[ 0]                     NULL              00000000  000000  000000  00  0  0  0
[ 1] .far.text             PROGBITS          00010000  00c0f2  004a78  00  AX  0  0  2
[ 2] .text                PROGBITS          00003100  0000f4  00976a  00  AX  0  0  2
[ 3] .rodata              PROGBITS          0000c870  009864  0035fd  00  A   0  0  4
[ 4] .data                PROGBITS          00001100  00ce62  000158  00  WA  0  0  2
[ 5] .bss                 NOBITS           00001250  00cfb2  001648  00  WA  0  0  2
[ 6] .noinit              NOBITS           00002898  00cfb2  000002  00  WA  0  0  2
[ 7] .vectors             PROGBITS          0000ffc0  00cfb2  000040  00  AX  0  0  1
[ 8] .comment             PROGBITS          00000000  011a6a  000030  01  MS  0  0  1
[ 9] .debug_aranges       PROGBITS          00000000  011aa0  000308  00  0   0  0  8
[10] .debug_info          PROGBITS          00000000  011da8  001b26  00  0   0  0  1
[11] .debug_abbrev        PROGBITS          00000000  0138ce  0009bb  00  0   0  0  1
[12] .debug_line          PROGBITS          00000000  014289  001003  00  0   0  0  1
[13] .debug_frame         PROGBITS          00000000  01528c  000270  00  0   0  0  4
[14] .debug_str           PROGBITS          00000000  0154fc  00046e  01  MS  0  0  1
[15] .debug_loc           PROGBITS          00000000  01596a  00162d  00  0   0  0  1
[16] .debug_ranges       PROGBITS          00000000  016f97  000108  00  0   0  0  1
[17] .gnu.attributes     LOOS+ffff5       00000000  01709f  000011  00  0   0  0  1
[18] .shstrtab            STRTAB            00000000  0170b0  0000c8  00  0   0  0  1
[19] .symtab              SYMTAB            00000000  0174c0  004770  10  20 385 4
[20] .strtab              STRTAB            00000000  01bc30  003eb0  00  0   0  0  1

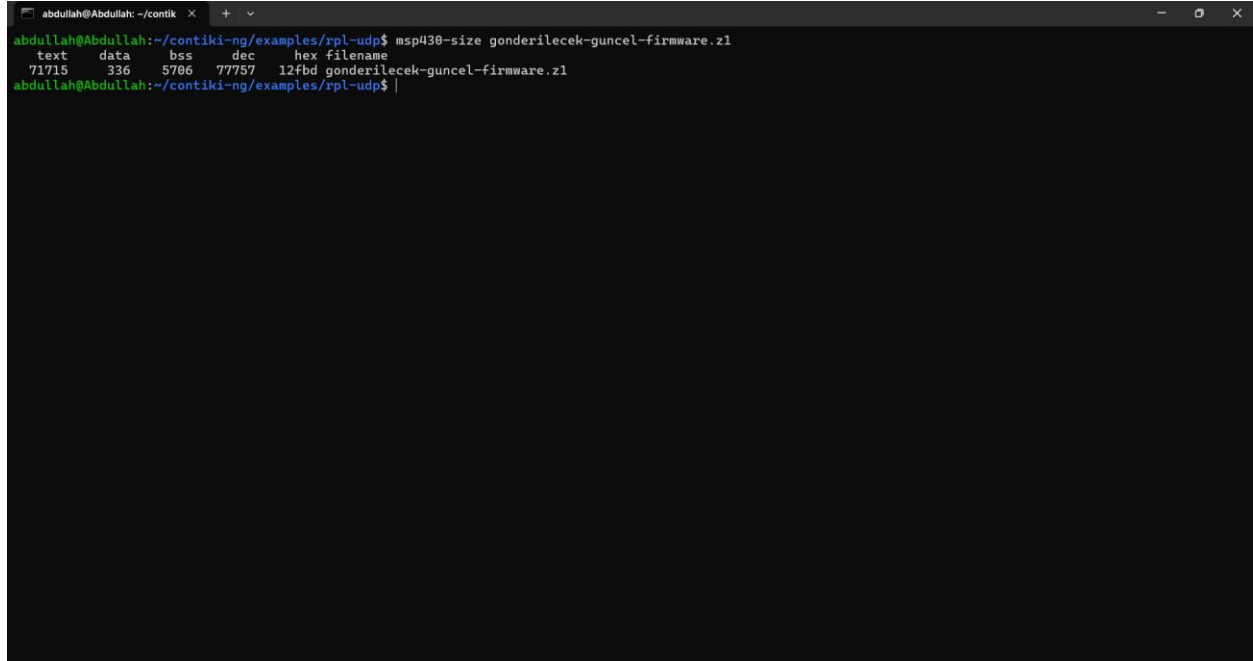
Key to Flags:
W (write), A (alloc), X (execute), M (merge), S (strings)
I (info), L (link order), G (group), T (TLS), E (exclude), x (unknown)
O (extra OS processing required) o (OS specific), p (processor specific)
abdullah@Abdullah:~/contiki-ng/examples/rpl-udp$
```

Section çıktısı firmware imajının bellekte hangi mantıksal bölümlerden oluştuğunu göstermektedir. `.text` bölümü çalıştırılacak kodu, `.data` ve `.bss` bölümleri değişkenleri, `.vectors` bölümü ise başlangıç ve kesme vektörleriyle ilişkili bilgileri temsil etmektedir.

11.3. Boyut Analizi

```
msp430-size
```

komutu ile firmware dosyasının kod ve veri boyutları incelenmiştir.



```
abdullah@Abdullah: ~/contik
abdullah@Abdullah:~/contiki-ng/examples/rpl-udp$ msp430-size gonderilecek-guncel-firmware.z1
text  data  bss   dec   hex filename
71715 336    5706  77757 12fbd gonderilecek-guncel-firmware.z1
abdullah@Abdullah:~/contiki-ng/examples/rpl-udp$ |
```

Boyut analizi sonucunda firmware imajının program kodu ve veri alanlarının bellekte ne kadar yer kapladığı görülmektedir. Bu bilgi, gömülü sistemlerde sınırlı flash ve RAM alanı kullanıldığı için önemlidir.

11.4. Sembol Analizi

```
msp430-nm -n
```

veya

```
msp430-readelf -s
```

komutu ile firmware dosyasındaki semboller incelenmiştir.

```
abdullah@Abdullah: ~/contiki x + v
abdullah@Abdullah:~/contiki-ng/examples/rpl-udp$ msp430-nm -n gonderilecek-guncel-firmware.z1
U button_hal_button_count
U button_hal_buttons
U gpio_hal_arch_init
U gpio_hal_arch_port_clear_pin
U gpio_hal_arch_port_interrupt_enable
U gpio_hal_arch_port_pin_cfg_set
U gpio_hal_arch_port_pin_set_input
U gpio_hal_arch_port_read_pin
U gpio_hal_arch_port_read_pins
U gpio_hal_arch_port_set_pin
U gpio_hal_arch_port_write_pin
U gpio_hal_arch_port_write_pins
U spi_arch_has_lock
00000000 A __IE1
00000000 T __far_bss_end
00000000 A __far_bss_size
00000000 T __far_bss_start
00000000 A __far_data_size
00000000 T __far_data_start
00000000 T _efar_data
00000000 A _far_end
00000001 A __IE2
00000002 A __IFG1
00000003 A __IFG2
00000006 A __UC1IF
00000007 A __UC1IFG
00000010 A __P3REN
00000011 A __P4REN
00000012 A __P5REN
00000013 A __P6REN
00000014 A __P7REN
00000014 A __PAREN
00000015 A __P8REN
00000018 A __P3IN
00000019 A __P3OUT
0000001a A __P3DIR
0000001b A __P3SEL
0000001c A __P4IN
0000001d A __P4OUT
0000001e A __P4DIR
```

Sembol tablosu, firmware içindeki fonksiyonların ve değişkenlerin adreslerini göstermektedir. Bu bilgiler firmware dosyasının linker tarafından belleğe nasıl yerleştirildiğini anlamak için kullanılmıştır.

12. 3. Aşama Hakkında Açıklama

Bu çalışmada 3. aşama olan CC1352R gerçek donanım üzerinde bootloader, CCFG alanı, gerçek flash yerleşimi ve reboot işlemi gerçekleştirilmemiştir. Çalışma kapsamı Cooja simülatörü üzerinde OTA firmware aktarımı ve gönderilecek firmware dosyasının ELF analizi ile sınırlandırılmıştır.

Şartnamede de 1. aşama için alıcı düğüm tarafından kaydedilen makine kodunun Cooja ortamında boot edilecek başlangıç adresi olarak kaydedilemeyeceği ve reboot işleminin beklenmediği belirtilmiştir. Bu nedenle bu raporda aktarımın başarıyla tamamlanması, firmware'in alıcı tarafta saklanması ve checksum ile doğrulanması üzerinde durulmuştur.

13. Sonuç

Bu projede Contiki-NG ve Cooja simülatörü kullanılarak OTA firmware aktarımı gerçekleştirilmiştir. Gönderici düğümde bulunan gonderilecek-guncel-firmware.z1 dosyası küçük bloklara ayrılmış ve UDP üzerinden alıcı düğüme gönderilmiştir.

Her blok için blok numarası, veri uzunluğu ve checksum bilgisi kullanılmıştır. Alıcı düğüm gelen blokları kontrol etmiş, doğru blokları flash benzeri kalıcı alana yazmış ve her doğru blok için ACK mesajı göndermiştir. Aktarım sonunda 16220 bloğun tamamı alınmış, 129760 byte firmware verisi saklanmış ve tüm imaj checksum değeri beklenen değere aynı çıkmıştır.

Böylece firmware dosyasının kablosuz ağ üzerinden kontrollü, sıralı ve güvenilir şekilde aktarılabilirdiği gösterilmiştir. Ayrıca gönderilen firmware dosyasının ELF yapısı araç zinciri komutlarıyla incelenmiş ve dosyanın ham veri değil, MSP430 mimarisi için hazırlanmış çalıştırılabilir firmware imajı olduğu açıklanmıştır.